RECURSIVE SELF-IMPROVEMENT IN AI

A Deep-Research Report on the Systems Teaching Machines to Rewrite, Retrain, and Redesign Themselves â€” and a Verdict on the Best One Yet

---

Prepared for: Akshan Â· Founder, Silisleek
Scope: Academic RSI (GÃ¶del-machine lineage), applied algorithm discovery, self-adapting weights, self-play reasoning, and frontier-lab production RSI evidence (Anthropic, OpenAI, DeepMind).
Method: Multi-pass web research across arXiv, lab technical reports, and primary-source blog posts (Sept 2026), cross-checked across independent sources before synthesis.

This report does not summarize hype. It traces one continuous engineering lineage â€” DGM â†’ SICA â†’ Huxley-GÃ¶del Machine â†’ Red Queen GÃ¶del Machine â€” the academic frontier of agents that literally rewrite their own source code, alongside AlphaEvolve, SEAL, and Absolute Zero, which self-improve different layers of the stack (algorithms, weights, and training data respectively). It closes with primary evidence from inside Anthropic and OpenAI on what recursive self-improvement looks like when it leaves the lab and starts shipping production code.

Â§0 Executive Summary

"Recursive self-improvement" (RSI) is used loosely across the field, so this report first separates **bounded self-refinement** (a system polishing one output, then stopping) from **true RSI** (a system whose changes persist, compound, and improve its own future ability to improve). A 1,250-paper 2026 arXiv survey found that almost everything marketed as "self-improving AI" is the former; genuine open-ended RSI remains bounded by three hard constraints on every measured system to date: it needs an external grounding signal (a verifier or benchmark), it is vulnerable to reward hacking and model collapse if that signal is gamed, and it is compute-bounded rather than intelligence-bounded.

Inside that boundary, one lineage of systems has made the most unambiguous, benchmarked progress: agents that rewrite their own Python codebase and validate each rewrite empirically. It runs **Darwin GÃ¶del Machine (2025) â†' Self-Improving Coding Agent (2025) â†' Huxley-GÃ¶del Machine (2025) â†' Red Queen GÃ¶del Machine (2026)** â€" each one explicitly built to beat its predecessor, and each one doing so on the same public benchmarks (SWE-bench Verified, Polyglot). As of September 2026, the **Red Queen GÃ¶del Machine (RQGM)** is the most recent, most rigorously validated entrant in this lineage and the closest thing the field has to a documented, reproducible "best self-improving AI system."

Running in parallel, three systems self-improve a different layer of the stack rather than their own agent scaffold: **AlphaEvolve** (Google DeepMind) evolves algorithms and has already been redeployed into Google's own production infrastructure and genomics pipelines; **SEAL** (MIT) teaches a language model to write its own fine-tuning data and permanently update its own weights; and **Absolute Zero** teaches a reasoner to invent its own training tasks with zero human-curated data. None of the three modifies its own source code the way the GÃ¶del-machine lineage does, which is why they are complementary to it, not competitors for the same title.

The largest-scale evidence of RSI, however, is not a single paper â€" it is what Anthropic and OpenAI report happening inside their own frontier-model pipelines. As of May 2026, Anthropic reports that **over 80% of the code merged into Anthropic's own codebase is written by Claude**, that Claude-run agents recovered 97% of an open-ended AI-safety research gap that two human researchers only closed 23% of in the same time, and that engineers now ship roughly 8x the code per quarter versus 2021-2025. OpenAI has separately stated that GPT-5.3-Codex was its "first model instrumental in creating itself." This is not a closed self-improvement loop yet â€" humans still choose research directions and score results â€" but at the scale of full frontier-lab R&D; pipelines, it is the most consequential RSI signal that currently exists.

Bottom line: There is no single AI system that is simultaneously the most autonomous, most general, and most impactful self-improver â€" those three properties currently trade off against each other. Section 6 gives a full reasoned verdict; the short version is that the Red Queen GÃ¶del Machine is the best-documented pure RSI system (most autonomous, narrowest domain), AlphaEvolve is the best real-world-deployed self-improving system (verified production impact, human-gated), and Anthropic's Claude-in-the-loop pipeline is the best evidence of RSI at civilizational scale (largest impact, least architecturally "closed").

Â§1 What "Recursive Self-Improvement" Actually Means

Because the term is used to describe everything from a chatbot re-reading its own answer to a hypothetical machine that redesigns its own successor, this report anchors on the taxonomy used in the most rigorous 2026 review of the field (*"The Last AI Built by Humans,"* alphaXiv 2609.11873), which scores systems on five increasing levels of autonomy:

L1 â€” Executing prescribed improvements	A human specifies exactly what to change; the AI just does it (e.g., an AI implementing a human-designed optimizer).
L2 â€” Selecting improvement strategies	The AI chooses which of several human-designed improvement strategies to apply.
L3 â€” Acquiring future learning experience	The AI decides what new experience or data it needs, and goes and gets it (self-play, task generation).
L4 â€” Adapting through deployment feedback	The AI updates itself continuously from real-world / environmental feedback, not just a fixed offline benchmark.
L5 â€” Improving the mechanisms that govern future improvement	The AI modifies its own improvement process itself â€” e.g. co-evolving the evaluator that judges its own changes.

Almost every system marketed as "self-improving" (self-refinement loops, agentic retries, RAG that logs corrections) sits at **L1-L2**. The systems worth taking seriously as genuine RSI candidates â€” the subject of this report â€— operate at **L3 to L5**: they generate their own training signal, adapt from live feedback, or evolve the very evaluator that scores them.

A second, orthogonal axis matters just as much: **what layer of the system actually changes.** A system can self-improve its *prompts/scaffold*, its *source code*, its *weights*, its *training data*, or its *evaluator*. The systems in this report are grouped by which layer they target, because that is what makes them complementary rather than directly comparable.

Â§2 The Core RSI Lineage: Agents That Rewrite Their Own Code

The theoretical "GÃ¶del machine" (Schmidhuber, 2003) imagined a self-improving AI that would only rewrite itself when it could formally *prove* the rewrite was beneficial. That proof requirement made it unbuildable. Every system below is a pragmatic descendant: instead of a proof, each uses **empirical validation against a coding benchmark** as the gate for keeping a self-modification. This is the most direct, most rigorously benchmarked RSI lineage that exists today, and each entry was explicitly built and published to outperform the one before it.

Darwin GÃ¶del Machine (DGM)

The system that proved code-rewriting RSI works at all

Layer: Own source code | **Autonomy:** L3-L4 | **Paper:** arXiv 2505.22954

DGM is a coding agent that reads and edits its own Python codebase, then empirically checks whether the edited version is actually better at solving coding problems â€” no formal proof, just a benchmark score. Instead of evolving one lineage, it keeps a growing **archive** of every agent variant ever produced and samples from the whole archive to breed the next generation, deliberately preserving "stepping-stone" agents that aren't the best performers yet but unlock better descendants later â€” the paper's core "open-endedness" insight.

How it self-improves:

- â€¢ Foundation model (LLM) proposes a code diff to the agent's own scaffold (e.g., a new file-editing tool, a better long-context strategy).
- â€¢ The modified agent is run against a coding benchmark (SWE-bench Verified, Polyglot).
- â€¢ If it scores higher, the new agent is added to the archive as a candidate parent for future generations.
- â€¢ Repeat for 80 iterations with no further human input.

Verified results:

- â€¢ Improved autonomously from 20.0% â†’ 50.0% on SWE-bench Verified over 80 iterations, with zero plateau versus a non-self-improving baseline.
- â€¢ Improved from 14.2% â†’ 30.7% on Polyglot, surpassing the human-engineered Aider agent on that benchmark.
- â€¢ Discovered its own improvements independently â€” better patch-validation steps, better long-context file handling, peer-review-style self-checks.

Limits / open problems:

- â€¢ A single full run takes about two weeks and significant API spend.
- â€¢ Still fell short of the best closed-source, human-engineered SWE-bench solutions at publication time.
- â€¢ Self-modification is limited to the agent's prompts/scaffold code, not the underlying LLM's weights.

Where it sits: The foundational proof-of-concept for the entire lineage below. Landmark, but already surpassed on every benchmark it introduced.

Self-Improving Coding Agent (SICA)

The parallel-track sibling that focused on single-lineage depth over archive breadth

Layer: Own source code | **Autonomy:** L3 | Cited throughout the DGM/HGM lineage as the key single-agent baseline

SICA modifies its own codebase the same way DGM does, but instead of growing a branching archive of many agent lineages, it keeps improving a single, best-known agent â€” closer to classic greedy hill-climbing than DGM's evolutionary tree. It is the direct baseline that later work (HGM) had to beat to justify a more expensive, tree-based search.

How it self-improves:

- â€¢ An LLM inspects the current best agent's own source and proposes one self-edit.
- â€¢ The edit is validated on the same coding benchmarks used by DGM.
- â€¢ Only the single best-performing lineage is carried forward â€” no archive of alternates.

Verified results:

- â€¢ Established that even single-lineage, non-branching self-modification produces real, compounding gains.
- â€¢ Later shown by both DGM and HGM to plateau earlier than archive/tree-based search, motivating the next two systems.

Limits / open problems:

- â€¢ Prone to getting stuck in local optima since it discards non-improving branches instead of banking them as future stepping stones.
- â€¢ No mechanism to identify agents that look mediocre now but unlock bigger future gains.

Where it sits: Historically important control condition; effectively superseded by HGM, which formalized exactly why single-lineage search underperforms.

Huxley-GÃ¶del Machine (HGM)

Fixed DGM's central flaw: the best-scoring agent isn't always the best parent

Layer: Own source code | **Autonomy:** L4 | **Paper:** arXiv 2510.21614 | Code: github.com/metauto-ai/HGM

HGM's contribution is a genuinely new idea, not just more compute: it identifies the **"Metaproductivityâ€“Performance Mismatch"** â€” the empirical finding that an agent's current benchmark score is a poor predictor of how good its *descendants* will be. Named after Thomas Huxley's concept of a biological "clade," HGM estimates a metric called CMP (clade metaproductivity): instead of asking "is this agent good?", it asks "how good, on average, are the children this agent tends to produce?" and uses that to decide which agent in the archive to breed from next.

How it self-improves:

- â€¢ Builds the same kind of self-modification archive/tree as DGM.
- â€¢ Estimates each node's CMP â€” the aggregate future performance of its descendant lineage â€” rather than its own raw score.
- â€¢ Uses CMP as the guide for tree search, provably approximating how the theoretical (provably-optimal) GÃ¶del machine would behave under stated assumptions.

Verified results:

- â€¢ Outperformed both DGM and SICA on SWE-bench Verified and Polyglot while using fewer CPU-hours.
- â€¢ An HGM-discovered agent reached human-level performance on SWE-bench Lite, matching the best officially-verified human-engineered coding agent (SWE-agent) under matched compute budgets.
- â€¢ Demonstrated strong transfer: an agent tuned on SWE-bench Verified with GPT-5-mini kept human-level performance when tested on a different benchmark (SWE-bench Lite) with a different backbone model (GPT-5).

Limits / open problems:

- â€¢ CMP is an estimated, not exact, quantity â€” its accuracy is itself a bottleneck.

â€¢ Still evaluated against a single, fixed benchmark criterion throughout a run â€” which the next system in the lineage identifies as the next weak point.

Where it sits: The first system in this lineage to match human-engineered, officially-verified agent performance â€” the point where RSI research crossed from "promising" to "competitive with the best humans have hand-built."

Red Queen GÃ¶del Machine (RQGM) â€” current published state of the art

Solves the last big loophole: agents were still being graded by a judge that never changes

Layer: Own source code + its own evaluator | **Autonomy:** L5 | **Paper:** arXiv 2606.26294

Every system above shares one hidden assumption: the yardstick used to judge whether a self-modification is an improvement (a fixed benchmark or verifier) never itself changes. RQGM's authors point out this is unrealistic â€” in real evolution, environments and competitors adapt too (the "Red Queen effect": you have to keep improving just to stay in the same relative place). RQGM makes the **evaluator itself part of the evolving system**: agents and the code-review judges that score them co-evolve together, epoch by epoch, which is exactly the L5 capability defined in Section 1 â€” improving the mechanism that governs future improvement, not just the thing being improved.

How it self-improves:

â€¢ Search is split into epochs; within an epoch the evaluation criterion is held fixed (preserving DGM/HGM-style improvement guarantees).

â€¢ At each epoch boundary, the evaluator (an evolved "agent-as-a-judge" code reviewer) is itself allowed to evolve â€” becoming cheaper and more discriminating.

â€¢ The reviewer signal is deliberately cheaper to compute than a full benchmark pass, so the system reaches a target quality level using far less compute than re-running the full benchmark every time.

Verified results:

â€¢ Exceeded HGM's held-out pass rate on the Polyglot benchmark while using 1.35x-1.72x fewer search tokens to reach the same quality.

â€¢ Showed that adding a co-evolved, cheap code-review signal is not just faster but strictly more accurate at the same compute budget than the prior best static-evaluator approach.

â€¢ Demonstrated evaluator replacement mid-run without collapsing the improvement guarantees that HGM and DGM rely on.

Limits / open problems:

â€¢ Still bounded to the coding domain â€” has not been shown to generalize RSI to open-ended, non-verifiable tasks.

â€¢ Co-evolving the judge alongside the agent reopens (in a controlled, epoch-bounded way) the exact reward-hacking risk that a fixed external evaluator was originally meant to prevent â€” the paper's epoch design is explicitly an engineering answer to that risk, not a total elimination of it.

â€¢ As with its predecessors, a full run still requires substantial, non-trivial compute and sandboxed infrastructure.

Where it sits: The most recently published, most rigorously validated entry in the direct DGMâ†'SICAâ†'HGMâ†'RQGM lineage. On the specific, well-defined criterion of "an AI system that autonomously rewrites its own code AND the standard used to judge those rewrites, with published, reproducible gains over every predecessor," RQGM is the current state of the art.

Â§3 Self-Improvement at Other Layers: Algorithms, Weights, and Data

These three systems are not competing for the same crown as the GÃ¶del-machine lineage â€” they self-improve a different part of the stack, and two of them (AlphaEvolve, SEAL) are arguably more consequential in practice because they change either deployed production code or the model's actual weights, rather than an experimental research agent's own scaffold.

AlphaEvolve

Self-improving algorithm discovery, already redeployed into real production systems

Layer: Target algorithms/code (not its own agent code) | **Autonomy:** L3 | Built on Gemini Flash + Gemini Pro

AlphaEvolve is an evolutionary coding agent: given a starting program and an automatic scoring function, it uses an ensemble of Gemini models (Flash for breadth of ideas, Pro for depth) to evolve entire codebases, not just single functions â€” extending DeepMind's earlier FunSearch work. It doesn't rewrite its own agent code the way DGM does; instead it recursively improves *the algorithms and infrastructure Google itself runs on*, which is why it has already been redeployed rather than left as a research artifact.

How it self-improves:

- â€¢ Maintains a database of candidate programs plus their evaluation scores.
- â€¢ A prompt sampler builds rich prompts from the best-known programs in that database.
- â€¢ The LLM ensemble proposes new program variants; each is automatically scored and, if better, added back to the database â€” a direct evolutionary loop, conceptually a sibling of DGM's archive but applied to target algorithms rather than to AlphaEvolve's own scaffold.

Verified results:

- â€¢ Matched or beat the best known bounds on multiple open mathematical problems (e.g., improved the best-known lower bound for a sum/difference-of-sets problem from Î¸=1.14465 to Î¸=1.1584).
- â€¢ Discovered a faster algorithm for a component of Google's own data-center scheduling, which was deployed into production.
- â€¢ May 2026 update: applied to DeepConsensus (Google's DNA-sequencing error-correction model), cutting variant-detection errors by 30%.
- â€¢ May 2026 update: applied to the AC Optimal Power Flow grid-optimization problem, raising a trained model's rate of finding feasible solutions from 14% to over 88%.

Limits / open problems:

- â€¢ Requires the user to supply a machine-checkable scoring function up front â€” it cannot self-improve on problems without an automatic evaluator.
- â€¢ Improves target algorithms, not itself; it is not evolving its own reasoning process or scaffold the way the GÃ¶del-machine lineage is.
- â€¢ Human researchers still select which problems AlphaEvolve is pointed at.

Where it sits: The best-evidenced example of RSI-style evolutionary search creating **real, deployed, production value** outside a benchmark leaderboard â€” genomics, power grids, and Google's own infrastructure, not just papers.

SEAL (Self-Adapting Language Models)

Teaches a model to write its own training data and permanently rewire its own weights

Layer: Own weights | **Autonomy:** L3-L4 | **Paper:** arXiv 2506.10943

Most LLM "self-improvement" today is temporary: in-context learning that vanishes the moment the context window resets. SEAL instead gives the model the ability to write a **"self-edit"** â€" a natural-language instruction describing exactly how it should be fine-tuned (what synthetic training data to generate, what hyperparameters to use) â€" and then actually applies that edit via real gradient updates, producing a *persistent* change to the model's own weights, not just a longer prompt.

How it self-improves:

- â€¢ Outer loop (reinforcement learning): the model proposes candidate self-edits, each is applied, and the model is scored on downstream task performance; good self-edits are rewarded, training the model to generate better ones over time.
- â€¢ Inner loop (supervised fine-tuning): the chosen self-edit is used as the model's own training data, and standard gradient descent bakes it permanently into the weights.
- â€¢ The model directly parameterizes and controls its own adaptation process, rather than relying on an external adaptation module.

Verified results:

- â€¢ Boosted knowledge-assimilation QA accuracy from roughly one-third correct (no adaptation) to nearly half correct after a couple of self-edit rounds.
- â€¢ Beat both naive prompting and untrained self-edit baselines on few-shot reasoning generalization by a wide margin.

Limits / open problems:

- â€¢ Repeated self-edits can cause **catastrophic forgetting** â€" the paper's own continual-learning experiments show performance on earlier tasks degrading as new self-edits are layered on, with no built-in fix yet (replay, constrained updates, and representational superposition are named as open directions, not solved problems).
- â€¢ Independent critics note the standard comparison (self-adapting model vs. a model that never trains at all) inflates the apparent gain versus more realistic baselines.
- â€¢ Not yet demonstrated at frontier-model scale â€" published results are on smaller research-scale models.

Where it sits: The most credible published route to RSI at the **weights layer** rather than the code layer â€" genuinely different from the GÃ¶del-machine lineage, and the most likely candidate to eventually merge with it (an agent that rewrites both its scaffold and its weights).

Absolute Zero Reasoner (AZR)

Removes human-curated data from the loop entirely via self-play

Layer: Own training curriculum/data | **Autonomy:** L3 | **Paper:** arXiv 2505.03335

Reinforcement learning with verifiable rewards (RLVR) still depends on humans supplying a curated set of questions and answers. Absolute Zero removes that dependency: a single model plays both roles in a self-play loop â€" a **proposer** that invents its own reasoning tasks (deduction, abduction, induction, framed as code) and a **solver** that attempts them â€" with a code executor providing objective, ungameable ground truth for both roles.

How it self-improves:

- â€¢ Proposer generates a candidate task; a code interpreter validates it is well-formed and assigns a "learnability" reward (neither trivial nor unsolvable).
- â€¢ Solver attempts the validated task; the code executor checks the answer and provides a verifiable correctness reward.
- â€¢ Both roles are trained jointly via reinforcement learning, with zero external human-labeled data at any point.

Verified results:

- â€¢ Achieved state-of-the-art results on combined math + coding reasoning benchmarks with literally zero curated training examples.
- â€¢ Outperformed models trained on tens of thousands of expert-labeled, in-domain examples, despite operating fully out-of-distribution from any human dataset.

Limits / open problems:

- â€¢ Confined to domains where a code executor can provide an objective, verifiable reward â€” doesn't generalize automatically to open-ended or subjective tasks.
- â€¢ The authors themselves flag this as most relevant for a future where AI capability exceeds what human-set tasks can usefully teach â€” an argument for later relevance more than a solved current bottleneck.

Where it sits: The strongest existing demonstration that a model can bootstrap its own useful training curriculum from nothing â€” directly relevant to a future where the GÃ¶del-machine lineage runs out of human-authored coding benchmarks to be judged against.

Â§4 The Elephant in the Room: RSI at Frontier-Lab Scale

Every system above is a controlled research artifact, benchmarked in a sandbox. The largest, highest-stakes evidence of recursive self-improvement in September 2026 comes instead from what Anthropic and OpenAI report happening inside their own model-development pipelines â€” a distributed, human-supervised version of the same loop, running at the scale of an entire company's R&D.;

Anthropic: "When AI Builds Itself" (May 2026 institute report)

Anthropic's own account, corroborated with internal metrics rather than benchmark claims, is the single richest primary source found in this research pass:

- **Over 80% of code merged into Anthropic's production codebase is now authored by Claude** (up from low single digits before Claude Code's Feb 2025 launch); leadership has separately estimated 90%+ once scripts and experimental code are included.
- Engineers merged roughly **8x as much code per quarter** in Q2 2026 versus the 2021-2024 baseline.
- In a controlled experiment, Claude-powered agents were left to run an **open-ended AI safety research project** end-to-end ("can a weaker model reliably supervise a stronger one?") and recovered **97% of the measurable performance gap** over 800 cumulative agent-hours and ~$18,000 of compute â€” versus two human researchers recovering only ~23% of the same gap working for about a week.
- On a mid-course "what should happen next" judgment task inside real research sessions, Claude's suggested next step was rated better than the human researcher's actual choice **64% of the time** as of April 2026, up from 51% in November 2025.
- On a fixed "make this training code run faster" benchmark with the goal and success metric held constant, Claude went from a ~3x speedup (Opus 4, May 2025) to a ~52x speedup (Mythos Preview, April 2026) â€” a domain Anthropic explicitly calls "superhuman" for skilled human researchers working the same problem.
- In April 2026, Claude shipped over 800 fixes that cut a class of API errors by 1000x, work Anthropic's own engineer estimated would have taken a human about four years.

Anthropic is explicit that this is **not yet closed-loop RSI**: humans still choose which problems matter and define the scoring rubric in every one of these examples â€” the gap that separates today's systems from one capable of fully autonomously designing its own successor. But at the scale of an entire frontier lab's R&D; pipeline, this is the most consequential, most extensively measured RSI signal that currently exists anywhere, academic or corporate.

OpenAI: GPT-5.3-Codex

In February 2026, OpenAI shipped GPT-5.3-Codex describing it as **"our first model that was instrumental in creating itself"** â€” earlier versions of the model had helped debug its own training run. Independent analysis of the announcement (Future AGI, Aug 2026) cautions that this was a **bounded engineering assist judged by people**, not a scored, self-reseeding closed loop like RQGM's â€” i.e., an L1-L2 event by the taxonomy in Section 1, dressed in RSI language. The distinction matters: it illustrates exactly how easily "AI helped build the next AI" gets reported as "recursive self-improvement" when the actual mechanism is much more bounded.

Â§5 Side-by-Side Comparison

System	Layer changed	Autonomy	Best verified result	Domain scope
Darwin GÃ¶del Machine (2025)	Own agent code	L3-L4	SWE-bench 20%â†'50%; Polyglot 14.2%â†'30.7%	Coding (narrow)
SICA (2025)	Own agent code	L3	Baseline for HGM; single-lineage gains	Coding (narrow)
Huxley-GÃ¶del Machine (2025)	Own agent code	L4	Human-level on SWE-bench Lite	Coding (narrow)
Red Queen GÃ¶del Machine (2026)	Own agent code + own evaluator	L5	Beats HGM w/ 1.35-1.72x fewer tokens	Coding (narrow)
AlphaEvolve (2025-26)	Target algorithms/ production code	L3	30% fewer genomics errors; 14%â†'88% grid solution rate	Math, infra, genomics, energy
SEAL (2025)	Own weights	L3-L4	~33%â†'~50% QA acc. after self-edits	Research-scale LLM adaptation
Absolute Zero (2025)	Own training curriculum/data	L3	SOTA math+code reasoning, zero human data	Verifiable reasoning tasks
Claude / Anthropic pipeline (2026)	Company-wide R&D process	L2-L3 (human-gated)	>80% of merged code; 97% of research gap closed vs. 23% human	General software R&D (broad, but human-directed)

Autonomy levels use the L1-L5 scale defined in Section 1. "Domain scope" is deliberately separated from "autonomy" because they trade off in every row above: the most autonomous systems (RQGM) are also the narrowest, and the broadest-scope system (Anthropic's pipeline) is currently the least architecturally autonomous.

Â§6 Connecting the Dots: The Verdict

Combining every thread above into a single answer requires resisting the temptation to name one universal "winner" â€” the research itself shows why that would be a category error. Three genuinely different questions are being conflated whenever someone asks for "the best self-improving AI system," and each has a different, defensible answer as of September 2026:

If the question is "which system most closely realizes the original GÃ¶del-machine dream?"

â†’ The **Red Queen GÃ¶del Machine (RQGM)**. It is the most recent, most rigorously benchmarked entry in a direct, published, four-generation lineage (DGM â†’ SICA â†’ HGM â†’ RQGM) where each system was explicitly built to and did outperform its predecessor on identical, reproducible benchmarks. It is also the first in the lineage to operate at Level 5 autonomy â€” improving not just itself, but the evaluator that judges its own improvements â€” which is the single capability every earlier system in the lineage explicitly lacked. On the narrow, well-posed question of "pure" self-referential RSI, this is the current, citable state of the art.

If the question is "which system has done the most real-world good, already, outside a benchmark?"

â†’ **AlphaEvolve.** It is the only system in this report whose self-discovered improvements are already running in production outside the lab that built it: a redeployed data-center scheduling algorithm at Google, a 30% error reduction in a real genomics pipeline, and a >6x improvement in a power-grid optimization model's success rate. Autonomy-wise it is a step behind the GÃ¶del-machine lineage (it improves target algorithms rather than its own reasoning process, and a human still points it at each problem), but it is the strongest evidence that RSI-style evolutionary self-improvement creates value beyond a leaderboard.

If the question is "which system is having the largest total impact on the pace of AI development itself?"

â†’ **Claude, inside Anthropic's own R&D; pipeline** (with GPT-5.3-Codex inside OpenAI's as a close, less transparently-documented parallel). This is the least architecturally "closed" loop of anything in this report â€” a human still chooses what problem to work on in every documented case â€” but it is operating at a scale nothing else here approaches: an entire company's software output, not a sandboxed benchmark run. The 97%-of-research-gap-recovered result and the sustained 8x engineering-output multiplier are, in aggregate, the largest measured effect of AI improving AI development that currently exists in the public record.

So, if forced to name one:

The single system that best deserves the label "the best self-improving AI system yet" â€” weighing autonomy, rigor, reproducibility, and recency together, and treating real-world deployed impact as the tie-breaker over pure benchmark autonomy â€” is the Red Queen GÃ¶del Machine, as the most current, most self-referential, most independently-reproducible point in the only lineage of systems that literally rewrites its own source code and improves its own judge, generation after generation, with every claim checkable against a public arXiv paper and public benchmark numbers. AlphaEvolve is the strongest challenger specifically because it has already escaped the lab; Anthropic's internal pipeline is the strongest challenger specifically because of its sheer scale. But neither is *itself* the self-modifying artifact in the way RQGM, DGM, SICA, and HGM are â€” they are evidence that the RSI

paradigm generalizes, not instances of the sharpest version of it.

Â§7 What Still Stands Between Here and Genuine, Open-Ended RSI

- â€¢ **Grounding dependency.** Every system in this report â€” RQGM included â€” still needs an external, machine-checkable signal (a benchmark, a code executor, a verifier) to know whether a self-modification actually helped. None has demonstrated open-ended self-improvement on tasks without such a signal.
- â€¢ **Evaluator gaming / collapse dynamics.** The moment an evaluator becomes part of what's being optimized (as in RQGM, or in SEAL's repeated self-edits), the system risks exploiting quirks in its own judge rather than genuinely improving â€” RQGM's epoch-bounded design and SEAL's observed catastrophic forgetting are both, in different ways, symptoms of this same underlying risk.
- â€¢ **Compute, not intelligence, is currently the binding constraint.** DGM's full run takes ~2 weeks and meaningful API spend; Anthropic explicitly frames its own scenario planning around whether the bottleneck becomes compute/energy supply rather than model capability.
- â€¢ **The direction-setting gap.** Anthropic's own data shows Claude matching or beating human judgment on well-specified execution and even on some open-ended research *steps*, but choosing which problems are worth working on at all remains the clearest remaining human advantage across every system in this report, corporate or academic.
- â€¢ **Safety and control.** Anthropic's own framing is the most candid public statement from any lab on this point: full recursive self-improvement, if reached, could either compound alignment successes or compound rare misalignments as a system builds its own successor â€” and no organization in this report claims to have solved that question, only to be actively researching it.

Sources consulted include: arXiv 2505.22954 (DGM), 2510.21614 (HGM), 2606.26294 (RQGM), 2506.13131 / DeepMind AlphaEvolve technical report and blog (2025-2026), 2506.10943 (SEAL, MIT), 2505.03335 (Absolute Zero), arXiv 2607.07663 (1,250-paper RSI taxonomy survey), alphaXiv 2609.11873 ("The Last AI Built by Humans"), and Anthropic's "When AI Builds Itself" institute report (anthropic.com/institute/recursive-self-improvement, May 2026).